

CURSO

TEST AUTOMATION ENGINEER

FORMACIÓN INTEGRAL



Objetivo General del Curso

ANALIZAR LAS HERRAMIENTAS DE DISEÑO DE TEST AUTOMATION ENGINEER, DE ACUERDO A LAS APLICACIONES WEB, MÓVILES Y APIS.

Objetivo específico del Módulo

EXPLICAR HERRAMIENTAS DE INTEGRACIÓN DE PRUEBAS EN CI/CD,
DE ACUERDO A LAS APLICACIONES WEB, MÓVILES Y APIS.

Contenidos	
Objetivo General del Curso.....	2
Objetivo específico del Módulo	2
Módulo 10, HERRAMIENTAS DE INTEGRACIÓN DE PRUEBAS EN CI/CD	4
Capítulo 1: PRINCIPIOS DE INTEGRACIÓN Y ENTREGA CONTINUA	5
¿Qué es la Integración Continua (CI)?	5
¿Qué es la Entrega Continua (CD)?	6
Relación entre CI/CD y las pruebas automatizadas	6
Principios fundamentales de CI/CD	7
Beneficios de CI/CD para QA Automation	8
Ejemplo práctico: Pipeline básico CI/CD con GitHub Actions.....	8
Capítulo 2: AUTOMATIZACIÓN CON GITHUB ACTIONS Y GITLAB CI	9
GitHub Actions: Estructura y flujo básico	9
GitLab CI/CD: Arquitectura y flujo	10
Comparación entre GitHub Actions y GitLab CI.....	11
Variables de entorno y secretos	11
Prácticas recomendadas para testing automatizado en CI/CD.....	12
Caso práctico común.....	12
Capítulo 3: VARIABLES DE ENTORNO, EJECUCIÓN CONDICIONAL Y ORQUESTACIÓN DE TAREAS	14
Variables de entorno	14
Ejecución condicional	16
Orquestación de tareas.....	17
Buenas prácticas de orquestación:	19
Capítulo 4: ANÁLISIS DE FALLOS Y NOTIFICACIONES AUTOMÁTICAS.....	20
¿Qué es un fallo en un pipeline CI/CD?.....	20
Estrategias para el análisis de fallos	20
Notificaciones automáticas	21
Automatización de análisis de fallos.....	23
Capítulo 5: PIPELINE COMPLETO: TEST + BUILD + DEPLOY	24
Etapas de Construcción (Build).....	25
Etapas de Despliegue (Deploy)	26
Ejemplo de pipeline completo (GitHub Actions).....	28
Buenas prácticas	29

Módulo 10, HERRAMIENTAS DE INTEGRACIÓN DE PRUEBAS EN CI/CD



Capítulo 1: PRINCIPIOS DE INTEGRACIÓN Y ENTREGA CONTINUA

La Integración Continua (CI) y la Entrega Continua (CD) son prácticas fundamentales del desarrollo moderno de software, orientadas a mejorar la calidad, la velocidad y la estabilidad del ciclo de vida del producto. Estas prácticas permiten automatizar desde la integración del código hasta su despliegue en ambientes controlados, facilitando así la detección temprana de errores, la ejecución constante de pruebas automatizadas y la disponibilidad de versiones confiables para entrega.

¿Qué es la Integración Continua (CI)?

La Integración Continua es una práctica de desarrollo que consiste en fusionar frecuentemente los cambios de código realizados por distintos desarrolladores en un repositorio compartido (al menos una vez al día), seguido por la ejecución automática de pruebas para validar dichos cambios.

Objetivos:

- Detectar errores de integración tempranamente.
- Asegurar la compilación y validación constante del sistema.
- Automatizar la ejecución de pruebas unitarias, de integración y estáticas.
- Establecer una única fuente de verdad en el repositorio.
- Herramientas comunes: GitHub Actions, GitLab CI, Jenkins, CircleCI, Travis CI.

¿Qué es la Entrega Continua (CD)?

La Entrega Continua es la extensión natural de la Integración Continua. Su objetivo es garantizar que cada cambio que pase las pruebas automáticas esté listo para ser desplegado en producción en cualquier momento.

Objetivos:

- Tener un producto en estado “deployable” en todo momento.
- Automatizar las pruebas funcionales, de aceptación y regresión.
- Reducir el riesgo de errores en producción.
- Aumentar la frecuencia y confiabilidad de los despliegues.
- No confundir con Despliegue Continuo, que además automatiza la liberación a producción sin intervención humana.

Relación entre CI/CD y las pruebas automatizadas

Las pruebas automatizadas son el núcleo de los pipelines CI/CD. Estas pruebas se integran en varias fases:

Fase del pipeline	Tipo de prueba involucrada	Objetivo
Build	Pruebas unitarias, linters	Validar sintaxis y lógica básica
Test	Pruebas de integración, funcionales, API	Verificar componentes acoplados
Pre-release	Pruebas E2E, de regresión, de UI, carga	Validar el comportamiento completo
Deploy	Smoke tests, validaciones de entorno	Confirmar que el entorno desplegado funciona

Principios fundamentales de CI/CD

1.- Automatización total del pipeline

Todo el flujo, desde el commit hasta el deploy, debe ser gestionado por scripts versionados.

2.- Retroalimentación rápida

El sistema debe notificar inmediatamente los errores al desarrollador (fallo de build, fallos en pruebas, conflictos).

3.- Versionamiento y trazabilidad

Cada ejecución debe estar asociada a un commit, tag o build ID que permita rastrear la fuente del cambio.

4.- Ambiente controlado y replicable

Los entornos de integración y pre-producción deben ser consistentes, frecuentemente gestionados con contenedores (Docker) e infraestructura como código (IaC).

5.- Pruebas automatizadas como base

Los pipelines deben ser bloqueantes ante fallos de pruebas críticas. Esto asegura que solo versiones estables progresen.

6.- Integración frecuente

No esperar días para integrar cambios. Entre más frecuente la integración, menor será el costo de resolver conflictos.

Beneficios de CI/CD para QA Automation

- Disminuye el tiempo de detección y corrección de errores.
- Mejora la calidad del software antes de llegar a QA manual.
- Permite ejecutar pruebas en paralelo, mejorando cobertura.
- Facilita la integración de pruebas de seguridad (SAST/DAST) y performance.
- Reduce significativamente los tiempos de feedback en el ciclo Dev-Test.

Ejemplo práctico: Pipeline básico CI/CD con GitHub Actions

```
yaml

name: CI/CD Pipeline

on: [push]

jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Install dependencies
        run: npm install
      - name: Run unit tests
        run: npm test
      - name: Run E2E tests with Playwright
        run: npx playwright test
```

Este pipeline:

- Se activa en cada push.
- Instala dependencias.
- Ejecuta pruebas unitarias y E2E.

La adopción de CI/CD como parte de las competencias de un Test Automation Engineer permite asegurar calidad continua, disminuir retrabajos, y acelerar la entrega de valor. Para ser efectivo en este entorno, el QA debe comprender el pipeline completo, saber configurar pruebas automatizadas en diferentes etapas y colaborar activamente con DevOps.

Capítulo 2: AUTOMATIZACIÓN CON GITHUB ACTIONS Y GITLAB CI

GitHub Actions y GitLab CI son plataformas de automatización CI/CD que permiten definir flujos de trabajo (workflows/pipelines) para compilar, probar, y desplegar software automáticamente. Ambas integran de forma nativa con sus respectivos repositorios de código, facilitando la implementación de pruebas automatizadas dentro del ciclo de desarrollo continuo. Estas herramientas son fundamentales para los profesionales del testing, ya que permiten ejecutar pruebas de manera confiable en cada cambio de código.

GitHub Actions: Estructura y flujo básico

¿Qué es GitHub Actions?

Es la solución de GitHub para CI/CD. Permite ejecutar flujos de trabajo definidos como YAML en el repositorio, activados por eventos como push, pull_request, schedule, entre otros.

Componentes clave:

- Workflow: Archivo .yml que define el pipeline.
- Jobs: Conjuntos de pasos ejecutados en un runner.
- Steps: Acciones o comandos individuales dentro de un job.
- Actions: Tareas reutilizables empaquetadas (como instalar Node.js o subir artefactos).

Ejemplo: Automatización de pruebas en GitHub Actions

```
yaml

name: Run tests

on: [push, pull_request]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Instalar dependencias
        run: npm install
      - name: Ejecutar pruebas automatizadas
        run: npm test
```

Características destacadas:

- Integración directa con GitHub.
- Marketplace de Actions reutilizables.
- Soporte para runners autoservidos (self-hosted).

GitLab CI/CD: Arquitectura y flujo

¿Qué es GitLab CI/CD?

Es un sistema de integración y entrega continua nativo de GitLab, activado mediante el archivo `.gitlab-ci.yml`, que define los distintos jobs y stages a ejecutar.

Componentes clave:

- **Pipeline:** Proceso completo definido por etapas.
- **Stages:** Fases del pipeline (por ejemplo: build, test, deploy).
- **Jobs:** Tareas individuales en cada stage.
- **Runner:** Agente que ejecuta los jobs (puede ser compartido o específico).

Ejemplo: Pruebas automatizadas en GitLab CI

```
yaml

stages:
  - test

test_job:
  stage: test
  image: node:18
  script:
    - npm install
    - npm test
```

Ventajas clave:

- Ejecuta en contenedores por defecto (Docker).
- Flexibilidad en runners y entorno (bare metal, Kubernetes).
- Integración con repos, artefactos, variables de entorno, secretos.

Comparación entre GitHub Actions y GitLab CI

Característica	GitHub Actions	GitLab CI/CD
Definición de pipeline	.github/workflows/*.yaml	.gitlab-ci.yml
Eventos de activación	Push, PR, tags, cron, issue, etc.	Push, Merge Request, tag, cron, etc.
Infraestructura runner	GitHub hosted o self-hosted	GitLab shared runners o self-hosted
Marketplace de acciones	Amplio (GitHub Actions Marketplace)	Limitado (scripts personalizados)
Soporte contenedores	Manual vía Docker o actions específicas	Integración nativa con imágenes Docker
Visibilidad de ejecución	En la pestaña "Actions" del repo	En "CI/CD > Pipelines" del proyecto

Variables de entorno y secretos

Tanto GitHub como GitLab permiten inyectar información sensible (tokens, claves, configuraciones) a través de:

- **GitHub:** Settings > Secrets and variables
Accedidos como `process.env.MI_SECRET` en scripts.
- **GitLab:** Settings > CI/CD > Variables
Accedidos como variables de shell: `$MI_VARIABLE`.

Esto es fundamental para pruebas autenticadas, configuraciones de entorno, despliegues controlados o conexión con APIs externas.

Prácticas recomendadas para testing automatizado en CI/CD

- Separar entornos: definir jobs distintos para test, build y deploy.
- Bloqueos por errores de test: detener el pipeline si fallan las pruebas unitarias o de integración.
- Paralelizar tests: dividir suites grandes en múltiples jobs.
- Persistir resultados: usar artefactos para almacenar reportes de test (como HTML o JSON).
- Etiquetar versiones: aplicar tags automáticos si el pipeline finaliza exitosamente.
- Uso de matrices: ejecutar tests en múltiples versiones de Node, navegadores o dispositivos.

Caso práctico común

Automatización de pruebas con Playwright en GitHub Actions:

```
yaml

name: Playwright Tests

on: push

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - name: Instalar dependencias
        run: npm ci
      - name: Instalar Playwright browsers
        run: npx playwright install
      - name: Ejecutar pruebas E2E
        run: npx playwright test
```

El dominio de GitHub Actions y GitLab CI como plataformas de automatización permite a un Test Automation Engineer incorporar las pruebas automatizadas en el corazón del desarrollo moderno. Esto asegura calidad continua, despliegues confiables, y un flujo de trabajo colaborativo entre desarrollo, QA y operaciones.

Capítulo 3: VARIABLES DE ENTORNO, EJECUCIÓN CONDICIONAL Y ORQUESTACIÓN DE TAREAS

En los pipelines de CI/CD modernos, la configuración flexible es esencial para adaptar las ejecuciones a distintos entornos (desarrollo, testing, producción), condiciones (tags, ramas, resultados previos), y tareas interdependientes. Para lograrlo, se utilizan tres mecanismos fundamentales:

- Variables de entorno para parametrizar comportamientos.
- Ejecución condicional para controlar cuándo se ejecuta una tarea.
- Orquestación de tareas para definir dependencias, paralelismo y flujo del pipeline.

Variables de entorno

¿Qué son?

Son pares clave-valor que se inyectan en tiempo de ejecución dentro de los jobs, y permiten modificar el comportamiento de scripts o herramientas sin alterar el código fuente.

Usos comunes:

- Definir entornos (NODE_ENV=production)
- Autenticación (API_KEY, TOKEN)
- URLs, rutas o configuraciones por ambiente
- Flags de activación (RUN_TESTS=true)

En GitHub Actions:

Definición local:

```
yaml
env:
  NODE_ENV: production
```

Definición en job o paso:

```
yaml
jobs:
  test:
    env:
      API_KEY: ${ secrets.API_KEY }
```

En GitLab CI:

Definición en archivo .gitlab-ci.yml:

```
yaml
variables:
  NODE_ENV: production
```

Definición vía interfaz:

Settings > CI/CD > Variables

Acceso:

GitHub: process.env.NOMBRE en Node.js.

GitLab: \$NOMBRE directamente en shell.

Ejecución condicional

¿Qué es?

Es la capacidad de ejecutar un job o paso solo si se cumple cierta condición, como una rama específica, la presencia de un tag, el resultado de otro job, o un valor booleano.

Condiciones típicas:

Ejecutar solo en main:

GitHub:

```
yaml

if: github.ref == 'refs/heads/main'
```

GitLab:

```
yaml

only:
  - main
```

Ejecutar si otro job fue exitoso:

GitHub:

```
yaml

if: success()
```

GitLab:

```
yaml

needs:
  - job: build
    optional: true
```

Ejecutar según variable:

GitHub:

```
yaml  
  
if: env.RUN_E2E == 'true'
```

GitLab:

```
yaml  
  
rules:  
  - if: '$RUN_E2E == "true"'
```

Condiciones inversas (GitHub):

```
yaml  
  
if: failure()  
if: always()
```

Orquestación de tareas

¿Qué significa?

Es el diseño del flujo del pipeline indicando:

- Qué tareas se ejecutan en paralelo.
- Qué tareas deben esperar a otras.
- Qué etapas se agrupan y cómo se distribuyen.

En GitHub Actions:

Por defecto, jobs son paralelos.

Se usa needs para definir dependencias:

```
yaml

jobs:
  build:
    ...
  test:
    needs: build
    ...
```

En GitLab CI:

Uso de stages para definir orden secuencial:

```
yaml

stages:
  - build
  - test
  - deploy
```

Uso de needs: para paralelizar dentro de un stage:

```
yaml

test:
  stage: test
  needs: [build]
```

Buenas prácticas de orquestación:

- Definir claramente stages por tipo de tarea.
- Minimizar dependencias innecesarias.
- Ejecutar pruebas unitarias antes que E2E.
- Dividir pruebas en matrices o jobs paralelos.

Ejemplo completo (GitHub Actions)

```
yami

name: Pipeline CI/CD

on: push

env:
  RUN_E2E: true

jobs:
  build:
    runs-on: ubuntu-latest
    steps:
      - run: echo "Build iniciado..."

  unit_tests:
    needs: build
    runs-on: ubuntu-latest
    steps:
      - run: echo "Ejecutando pruebas unitarias"

  e2e_tests:
    needs: build
    if: env.RUN_E2E == 'true'
    runs-on: ubuntu-latest
    steps:
      - run: echo "Ejecutando pruebas E2E"
```

El manejo adecuado de variables, condiciones y la orquestación de tareas permite a un Test Automation Engineer diseñar pipelines adaptables, eficientes y seguros. Estas técnicas aseguran que las pruebas se ejecuten en los momentos correctos, en los entornos adecuados, y con una lógica flexible que se adapta al flujo de trabajo real de los equipos de desarrollo.

Capítulo 4: ANÁLISIS DE FALLOS Y NOTIFICACIONES AUTOMÁTICAS

En entornos de Integración y Entrega Continua (CI/CD), los fallos durante el pipeline (builds rotos, pruebas fallidas, despliegues erróneos) deben ser detectados, analizados y reportados en tiempo real. Para asegurar una respuesta rápida y eficiente del equipo, se requiere una combinación de mecanismos automáticos de análisis de fallos y sistemas de notificación integrados. Esto es clave para mantener la calidad del software y la estabilidad del proceso de entrega.

¿Qué es un fallo en un pipeline CI/CD?

Un fallo en CI/CD puede ser:

- Error en la compilación o construcción del proyecto.
- Fallo en pruebas unitarias, funcionales, E2E, o de integración.
- Problemas en dependencias (paquetes rotos, versiones incompatibles).
- Error de configuración en el entorno de ejecución.
- Fallo en la autenticación con sistemas externos (APIs, repositorios).
- Error en scripts de despliegue o acceso a infraestructura.

Cada tipo de fallo requiere detección, clasificación, trazabilidad y resolución rápida.

Estrategias para el análisis de fallos

a. Registro detallado (Logs)

Los pipelines deben generar logs detallados por cada step, job y stage.

Es fundamental mantener logs de:

- Salida de consola.
- Errores de prueba.
- Stack traces y código de error.
- GitHub Actions y GitLab CI generan logs en tiempo real con marcado visual.

b. Captura de artefactos de error

- Guardar reportes de errores y resultados de test fallidos como artefactos descargables (por ejemplo, en formato XML, JSON, HTML).
- Playwright, Cypress, Jest, JUnit, entre otros, permiten generar reportes con información de los tests fallidos.

c. Integración de dashboards de análisis

- Se recomienda integrar herramientas de análisis como:
- Allure Report, JUnit Viewer, Test Reports en GitLab.
- Dashboards personalizados con Grafana, Prometheus, Datadog (para métricas de calidad).

d. Etiquetado y seguimiento de errores

Automatizar el etiquetado de issues si un test falla (por ejemplo: "test-flaky", "regression-failure").

Integración con herramientas de gestión de errores: Sentry, Bugsnag, Rollbar, etc.

Notificaciones automáticas

¿Qué son?

Mecanismos que envían mensajes automáticos al equipo cuando ocurre un evento relevante en el pipeline (por ejemplo, un fallo o una ejecución exitosa).

Canales comunes:

- Email (alertas a devs/QA).
- Slack, Microsoft Teams (mensajes automáticos en canales).
- Discord, Telegram, Mattermost.
- Webhooks a herramientas de monitoreo o gestión de proyectos (Jira, Trello).

En GitHub Actions (Slack):

```
yaml

- name: Slack Notification
  uses: slackapi/slack-github-action@v1.23.0
  with:
    payload: '{"text": "❌ Fallo en pruebas E2E. Revisión requerida."}'
  env:
    SLACK_WEBHOOK_URL: ${ secrets.SLACK_WEBHOOK }
```

En GitLab CI:

```
yaml

notify:
  stage: notify
  script:
    - curl -X POST -H 'Content-type: application/json' \
      --data '{"text": "🔥 Error en etapa de test."}' \
      $SLACK_WEBHOOK
  when: on_failure
```

Buenas prácticas:

- Notificar solo en eventos relevantes (fallos, regresiones, errores críticos).
- Personalizar el mensaje: incluir nombre del job, enlace al pipeline, rama, autor del commit.
- Evitar notificaciones excesivas (ruido): usar filtros, condiciones o canales dedicados.

Automatización de análisis de fallos

Herramientas útiles:

- Test reporters: Allure, Mocha Awesome, Jest HTML Reporters.
- Linters + SAST: ESLint, SonarQube, CodeQL.
- Visual regression: Percy, AppliTools, Chromatic.
- Alertas automáticas: GitHub Issues bot, GitLab Issue Bot, Sentry automation.

Ejemplo de integración en GitHub Actions:

```
yaml
- name: Ejecutar pruebas y capturar fallos
  run: |
    npm test || echo "Tests fallidos"
- name: Subir reporte HTML de fallos
  uses: actions/upload-artifact@v3
  with:
    name: test-report
    path: reports/html
```

Un Test Automation Engineer debe ser capaz de:

- Diseñar pipelines resilientes y autoexplicativos en caso de error.
- Integrar herramientas que generen reportes automáticos y visuales.
- Implementar notificaciones que alerten a tiempo al equipo correcto.
- Mantener trazabilidad de fallos con artefactos, logs y seguimiento automatizado.

Estas prácticas reducen el tiempo de respuesta ante errores, mejoran la colaboración y permiten mantener la calidad y estabilidad del software en ciclos de entrega acelerados.

Capítulo 5: PIPELINE COMPLETO: TEST + BUILD + DEPLOY

Un pipeline completo de CI/CD orquesta de forma automatizada las etapas críticas del ciclo de vida del software: pruebas, construcción del artefacto y despliegue. Esta práctica asegura que cualquier cambio en el código sea verificado, compilado y entregado en ambientes controlados (staging o producción), minimizando errores humanos, acelerando el feedback y aumentando la calidad del producto final.

Etapas de Testing (Test)

Objetivo:

Verificar que los cambios de código no introduzcan errores y que el sistema siga comportándose como se espera.

Tipos de pruebas integradas:

- Pruebas unitarias: verifican funciones individuales o componentes aislados.
- Pruebas de integración: validan la interacción entre varios módulos del sistema.
- Pruebas E2E: simulan el uso real del sistema, validando la experiencia completa del usuario.
- Pruebas de API: confirman que los endpoints funcionan con entradas y respuestas válidas.
- Pruebas de regresión: aseguran que funcionalidades anteriores siguen funcionando.

Herramientas comunes:

- Node.js: Jest, Mocha, Playwright, Cypress.
- Java: JUnit, TestNG, Selenium.
- Python: Pytest, Robot Framework.

Resultado:

- Informe de test (HTML, XML, JSON).
- Artefactos con reportes o capturas de errores.
- Fallos detienen la ejecución del pipeline.

Etapas de Construcción (Build)

Objetivo:

Generar un artefacto compilado, empaquetado y listo para ser ejecutado o desplegado.

Tareas típicas:

- Compilación del código (TypeScript → JavaScript, Java → Bytecode).
- Transpilación y minificación (en frontend).
- Generación de binarios o paquetes (.jar, .zip, .docker, .tar.gz).
- Verificación de dependencias.
- Firma o versionado del artefacto.

Tipos de artefactos:

- Imagen Docker (contenedor).
- Binario ejecutable.
- Librería empaquetada.
- Bundle web (dist/ en Angular/React/Vue).

Herramientas comunes:

- npm / webpack / babel (JavaScript)
- Maven / Gradle (Java)
- Docker / Podman (contenedores)
- Make / CMake (C/C++)

Etapa de Despliegue (Deploy)

Objetivo:

Liberar el artefacto en un entorno específico (QA, staging, producción), asegurando que esté operativo y configurado correctamente.

Modalidades de despliegue:

- Manual aprobado: requiere validación antes de ejecutar (aprobación QA/DevOps).
- Automático a entorno de staging: al pasar todas las pruebas.
- Automático a producción: en despliegue continuo (CD).
- Canary o blue/green: despliegues progresivos y reversibles.

Herramientas de despliegue:

- Docker + Docker Compose
- Kubernetes / Helm
- Heroku, AWS, Azure, GCP
- Ansible, Terraform (infraestructura como código)

Notas importantes:

- Usar variables de entorno para configurar diferencias entre entornos.
- Validar que el entorno esté sano (smoke tests post-deploy).
- Notificar al equipo después del despliegue.

Ejemplo de pipeline completo (GitHub Actions)

name: CI/CD Pipeline

on:

push:

branches:

- main

jobs:

test:

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3
- run: npm install
- run: npm test

build:

needs: test

runs-on: ubuntu-latest

steps:

- uses: actions/checkout@v3
 - run: npm run build
 - uses: actions/upload-artifact@v3
- with:
- name: build
 - path: dist/

deploy:

needs: build

runs-on: ubuntu-latest

if: github.ref == 'refs/heads/main'

steps:

- uses: actions/download-artifact@v3
- with:
- name: build
 - run: echo "Desplegando en servidor..."
 - run: scp -r dist/ usuario@servidor:/ruta/deploy

Buenas prácticas

- Ejecutar pruebas críticas antes de construir o desplegar.
- Bloquear despliegues si hay errores de test.
- Versionar los artefactos construidos.
- Implementar monitoreo post-despliegue (logs, métricas, alertas).
- Documentar los pipelines y mantenerlos bajo control de versiones.
- Usar ramas o entornos separados para producción y staging.

Un pipeline completo test + build + deploy permite entregar software de forma rápida, confiable y automatizada. Para un Test Automation Engineer, es vital entender cada etapa, saber cómo integrar pruebas automatizadas en el flujo, y colaborar estrechamente con DevOps para asegurar que solo versiones validadas lleguen a los entornos productivos.